

DogMatch Project

Relazione Strutturale

Analisi architetturale dei 4 repository

Maggio 2026

1. Visione d'insieme

DogMatch e' una piattaforma di dog-matching distribuita su 4 repository indipendenti che comunicano tramite API REST e WebSocket. L'architettura separa nettamente il layer AI, il backend applicativo, il frontend web e l'app mobile.

Architettura generale

```
[Frontend Web :5173]  --+
                      +---> [backend_node :3000] ---> [dogmatch_ai :8000 / GPT-4o]
[Mobile App (Expo)]  --+
                      |
                      [MySQL DB]
```

Porte e protocolli

Servizio	Porta	Protocollo	Note
dogmatch_ai (FastAPI)	8000	HTTP REST	Uso interno - non esposto al client
backend_node (Express)	3000	HTTP REST + WebSocket	Unico punto d'ingresso pubblico
Frontend-DogMatch (Vite)	5173	HTTP	Proxy /api -> localhost:3000
dogmatch-mobile (Expo)	--	HTTP -> 3000	Base URL configurata in .env

2. dogmatch_ai - Servizio AI (Python / FastAPI)

Ruolo e stack

Cervello del chatbot SnoutBot. Riceve messaggi dal backend Node.js, li elabora tramite GPT-4o con un sistema di tool-calling orchestrato e restituisce la risposta. I client non contattano mai questo servizio direttamente.

Stack: Python 3.11, FastAPI, OpenAI SDK, pytest

File principali

File	Responsabilita'
main.py	Entry point FastAPI - routing /chat e /chat/stream (SSE)
orchestrator.py	Loop tool-calling con GPT-4o
config.py	Variabili d'ambiente
database.py	Connessione al database
models.py	Pydantic models (request/response)
rate_limit.py	Protezione rate-limiting per utente
background.py	Task asincroni in background
knowledge_base.json	Dominio di conoscenza razze e salute cani

Tool AI disponibili (tools/)

Tool	Funzione
search_dogs.py	Ricerca cani nel database per razza, eta', taglia ecc.
memory.py	Memoria persistente della conversazione per utente
notifications.py	Invio notifiche push tramite backend
profile.py	Lettura profilo utente corrente
moderation.py	Filtraggio contenuti inappropriati

knowledge.py	Interroga la knowledge base locale
search_web.py	Ricerca web tramite Tavily API
booking.py	Gestione prenotazioni visite
activity.py	Tracciamento attivita' utente
executor.py	Esecutore orchestrato delle tool calls

Endpoint esposti

Endpoint	Metodo	Payload input	Risposta
POST /chat	POST	{message, user_id, context, history}	{risposta: string}
POST /chat/stream	POST	idem	SSE stream di token

Test suite (tests/)

- test_orchestrator.py - test del loop tool-calling
- test_integration.py - test end-to-end del flusso chat
- test_rate_limit.py - verifica limiti per utente
- test_notifications.py - invio e ricezione notifiche
- test_config.py - caricamento configurazione
- test_search_utils.py - funzioni di ricerca
- conftest.py - fixture condivise pytest

3. DogMatch_Project / backend_node - Backend Principale (Node.js / Express)

Ruolo e stack

Layer di orchestrazione centrale. Gestisce autenticazione JWT, accesso al database MySQL, logica di business e funge da proxy sicuro verso il servizio AI. Espone un'unica API REST ai client web e mobile.

Stack: Node.js, Express.js, Sequelize ORM, Socket.io, MySQL, JWT

Struttura MVC (src/)

```
src/
  controllers/  <- logica HTTP per ciascuna risorsa
  models/      <- modelli Sequelize (Utente, Cane, Messaggio...)
  routes/      <- definizione endpoint REST
  middleware/   <- auth JWT, validazione input, gestione upload
  services/     <- business logic complessa (ChatbotService, push)
  utils/       <- funzioni helper
  socket.js    <- configurazione Socket.io e stanze
```

Modelli Sequelize

Model	Tabella / Funzione
Utente.js	Utenti registrati - auth, privacy, profilo
Cane.js	Cani pubblicati - razza, eta', foto, pedigree
Interazione.js	Like/Dislike tra utenti (con campo 'intento')
Messaggio.js	Messaggi P2P tra utenti
MessaggioAI.js	Storico conversazioni con SnoutBot
Notifica.js	Notifiche push in-app
SegnalazioneUtente.js	Segnalazioni utenti per moderazione

Migrations (ordine cronologico)

Data	Migration
20260319	create-utenti, create-cani
20260319	create-messaggi-ai
20260324	create-interazioni, create-messages
20260504	add-intento-to-interazioni
20260504	fix-messaggi-paranoid-indexes
20260504	add-privacy-fields-to-utenti
20260504	fix-messaggi-ai-schema
20260506	fix-missing-columns
20260507	create-notifiche

Endpoint principali

Route	Controller	Note
POST /api/chat/bot	botController.js	Auth JWT - proxy a dogmatch_ai:8000
POST/GET /api/auth/*	authController.js	Login, register, refresh token

CRUD /api/cani/*	caneController.js	Gestione cani + upload foto
GET/POST /api/chat/*	chatController.js	Messaggi P2P
GET/PUT /api/utenti/*	utenteController.js	Profilo, privacy
GET/POST /api/notifiche/*	notificaController.js	Notifiche in-app
* /api/admin/*	adminController.js	Area admin protetta
GET/POST /api/interazioni/*	interazioneController.js	Like/match

WebSocket (Socket.io)

Il file socket.js configura le stanze per utente (room: user_<id>). L'evento bot_typing viene emesso durante l'elaborazione AI per mostrare l'indicatore di digitazione nei client in tempo reale.

4. Frontend-DogMatch - Web App (React / Vite)

Ruolo e stack

Interfaccia web utente. Vite proxying /api -> localhost:3000 evita problemi CORS in sviluppo. Autenticazione JWT memorizzata in localStorage.

Stack: React 18+, Vite, Redux Toolkit, axios, Socket.io-client

Pagine (src/pages/)

Pagina	Funzione
PrimaPage.jsx	Landing page pubblica
Login.jsx / Registrazione.jsx	Autenticazione utente
Home.jsx	Discovery cani con filtri
Messaggi.jsx	Chat P2P real-time
RichiesteMatch.jsx	Gestione richieste di match ricevute/inviare
UserProfilo.jsx	Profilo utente + cani pubblicati
ViewProfiloUtente.jsx	Visualizzazione profilo altrui
AdminPanel.jsx / AdminHome.jsx	Area admin - moderazione e statistiche

Componenti chiave (src/components/)

Componente	Funzione
SnoutBot.jsx	Widget chatbot floating - chiama POST /api/chat/bot
ChatWindow.jsx	UI messaggi con scroll e bubble
MappaRicerca.jsx	Mappa geografica per ricerca cani vicini
MatchAnimation.jsx	Animazione celebrativa al match confermato
NotificationDropdown.jsx	Notifiche in tempo reale via socket
Navbar.jsx / NavbarPubblica.jsx	Navigazione autenticata / pubblica
BarraRicercaRazza.jsx	Filtro ricerca per razza
AdminMatchSidebar.jsx	Sidebar admin per gestione match
modals/InfoCane.jsx	Modal dettaglio cane

Services, State e Hooks

File	Layer	Funzione
chatServices.js	Service	Chiamate API chat P2P e bot
dogServices.js	Service	Chiamate API gestione cani
interazioneServices.js	Service	Like, dislike, match
notificaServices.js	Service	Lettura e gestione notifiche
socketService.js	Service	Connessione Socket.io
store/index.js + dogReducer.js	Redux	Stato globale cani
hooks/useAuth.js	Hook	Autenticazione e token management

5. dogmatch-mobile - App Mobile (React Native / Expo)

Ruolo e stack

App mobile cross-platform (iOS e Android). Usa le stesse API del frontend web puntando a localhost:3000. TypeScript adottato parzialmente (hooks e servizi), mentre screens e navigation rimangono in JavaScript.

Stack: React Native, Expo SDK, TypeScript (parziale), Redux Toolkit, Expo Router

Screen principali (src/screens/)

Screen	Funzione
LoginScreen / RegisterScreen	Autenticazione
ForgotPasswordScreen	Recupero password
DiscoveryScreen	Swipe discovery cani - cuore dell'app
SnoutChatScreen	Chat con SnoutBot AI
ChatListScreen / MessagesScreen	Lista chat e messaggi P2P
ProfileScreen / EditProfileScreen	Profilo utente
NotificheScreen	Notifiche push e in-app
RichiesteRicevuteScreen	Richieste di match ricevute
AdminDashboard/Users/Dogs/Stats/ReportsScreen	Area admin mobile completa

Navigazione

Il progetto usa due sistemi di routing in parallelo: Expo Router (basato su file system, cartella `_app/`) per le tab principali, e AppNavigator.js (React Navigation stack) per il flusso auth e schermate complesse.

```
_app/
  _layout.tsx      <- Root layout Expo Router
  (tabs)/_layout.tsx <- Tabs bottom navigation
  (tabs)/index.tsx <- Home tab
  chat.tsx         <- Chat screen
  modal.tsx        <- Modal generico

src/navigation/
  AppNavigator.js  <- Stack navigator React Navigation
```

Custom Hooks (src/hooks/)

Hook	Linguaggio	Funzione
useChatbot.ts	TypeScript	Stato chat AI, chiamate /api/chat/bot, typing indicator socket
usePushNotifications.js	JavaScript	Registrazione token Expo push, ricezione notifiche

Redux Slices (src/store/slices/)

Slice	Stato gestito
authSlice.js	Token JWT, dati utente autenticato, stato login
interazioneSlice.js	Like, dislike, match in corso
notificheSlice.js	Lista notifiche, badge counter non lette

Services (src/services/)

File	Funzione
api.js	Istanza axios con base URL e interceptor JWT
adminApi.js	Endpoint admin (users, dogs, stats, reports)
chatApi.ts	Chiamate chat - tipizzato TypeScript
socket.js	Connessione Socket.io, stanze, eventi bot_typing
storage.js	AsyncStorage wrapper per token e preferenze

6. Flusso dati principale - Chat AI

1. Utente scrive messaggio (web o mobile)
2. Frontend -> POST /api/chat/bot [Authorization: Bearer <JWT>]
3. backend_node:
 - a. Verifica JWT (authMiddleware)
 - b. Carica profilo utente + cani + storico conversazione da MySQL
 - c. Costruisce system prompt contestualizzato
 - d. Emette socket 'bot_typing' -> room 'user_<id>' (typing indicator)
 - e. Proxy POST a dogmatch_ai:8000/chat
4. dogmatch_ai:
 - a. Riceve {message, user_id, context, history}
 - b. orchestrator.py avvia loop tool-calling con GPT-4o
 - c. Se necessario chiama tool: search_dogs, memory, knowledge, web...
 - d. Restituisce {risposta: string}
5. backend_node salva il messaggio in MessaggioAI (MySQL)
6. Risposta JSON restituita al client

7. Schema database - Relazioni principali

Utente	--<	Cane	(1 utente puo' avere piu' cani)
Utente	--<	Interazione	(like/dislike verso altri utenti)
Utente	--<	Messaggio	(chat P2P: mittente/destinatario)
Utente	--<	MessaggioAI	(storico SnoutBot per utente)
Utente	--<	Notifica	(notifiche push/in-app)
Utente	--<	SegnalazioneUtente	(moderazione contenuti)

8. Punti di attenzione architetturali

1. Duplicazione ChatWindow

Il componente e' presente sia in Frontend-DogMatch/src/components/ChatWindow.jsx che in dogmatch-mobile/components/Chatbot/ChatWindow.tsx. Eventuali fix devono essere replicati manualmente in entrambi i luoghi.

2. TypeScript parziale nel mobile

Mix di .js (screens, navigation, services base) e .ts/.tsx (hooks, chatApi). Manca una coverage uniforme che garantirebbe type-safety su tutto il layer dati.

3. Due sistemi di routing coesistenti

Expo Router (_app/) e React Navigation (AppNavigator.js) sono attivi simultaneamente. Rischio di comportamenti inconsistenti nel deep linking e nella navigazione back.

4. Upload file gestiti localmente

Le immagini di cani e utenti sono salvate in backend_node/uploads/ sul filesystem locale. Non c'e' uno storage cloud (S3 o equivalente): problema di scalabilita' e persistenza deploy.

5. AI service senza autenticazione propria

dogmatch_ai:8000 non verifica token in ingresso. E' sicuro solo se la porta e' vincolata alla rete locale (127.0.0.1). Non deve mai essere esposto pubblicamente.